

Programação Orientada a Objetos

Conceitos gerais:

- A Programação Orientada a Objetos (POO) é um paradigma de programação baseado no conceito de *objetos*, que podem conter tanto dados (campos, atributos ou propriedades) quanto código (procedimentos ou métodos).
 - As classes podem ser vistas como a evolução do conceito de tipo, enquanto os objetos são como se fossem uma evolução do conceito de variáveis.
- Os procedimentos de um objeto podem acessar e geralmente modificar os campos de dados do objeto ao qual estão associados (em C++, os objetos têm uma noção de si mesmos, através do ponteiro `this`).
- Em POO, os programas são projetados criando objetos que interagem entre si.
- A maioria das linguagens POO, inclusive C++, baseia-se em *classes* para construir objetos.
 - As classes são como se fossem uma evolução do conceito de `struct`.

Principais características da POO:

- Classes: definições para o formato dos dados e os procedimentos disponíveis para um determinado tipo de objetos com características em comum.
- Objeto: instâncias de classes. Os objetos às vezes correspondem a coisas encontradas no mundo real. Os objetos são uma expansão do conceito de variáveis.
 - Objetos fornecem uma camada de abstração que pode ser usada para separar o código interno (que integra a classe) do externo (de quem usa a classe).
- Encapsulamento: conceito de POO que une os dados e as funções que manipulam esses dados, mantendo os dois protegidos contra interferências externas e uso indevido.
 - Permite a *ocultação de dados*: proteção de outras partes do programa de modificações se a estrutura interna do objeto for alterada. A proteção envolve o fornecimento de uma interface estável que oculta os detalhes com maior probabilidade de alteração.
- Construção incremental de objetos: objetos maiores podem ser projetados baseando-se em objetos menores pré-existentes.
 - *Composição*: objetos podem conter outros objetos em seus campos de dados. A composição representa relações do tipo "contém-um" ou "é-composto-de" entre objetos.
 - *Herança*: permite que as classes sejam organizadas em uma hierarquia que represente relações do tipo "é-um-tipo-de" entre objetos.
- Polimorfismo: interface única para entidades de diferentes tipos ou o uso de um único símbolo para representar vários tipos diferentes.
 - Polimorfismo de função (ou *ad hoc*): funções que podem ser usadas com argumentos de tipos diferentes, mas que se comportam de maneira diferente, dependendo do tipo de argumento ao qual elas são aplicadas. Conhecido como sobrecarga de função ou sobrecarga de operador em C++.
 - Polimorfismo paramétrico: permite que uma função ou um tipo de dados seja escrito genericamente, para que ele possa manipular valores de maneira uniforme sem depender de seu tipo. Implementado através dos `template` em C++.
 - Polimorfismo por subtipagem: quando o código de chamada de uma função é independente de qual classe na hierarquia de herança executará a função - a classe pai ou um de seus descendentes. Permite que uma função seja escrita para receber como parâmetro um objeto de um determinado tipo T, mas também funcione corretamente se for passado um objeto que pertence a um tipo S, que é um subtipo de T (herda de T). Em C++, é implementado através dos métodos virtuais.

Classes C++

Tipos de membros das classes:

- Dados membro: similares aos existentes nas `struct`.
 - Funções membro (também denominados métodos).
 - Membros privados: só podem ser acessados pelos outros membros da classe
 - Membros públicos: podem ser acessados externamente à classe
 - Dados membro geralmente são privados.
 - Funções membro geralmente são públicas.
- ```
class MyClass {
private:
 int dado1; // Dado membro privado
 void func1(); // Função membro privada - não muito comum
public:
 double x; // Dado membro público - não muito comum
 void func2(); // Função membro pública
};
```
- Em C++, uma `struct` é equivalente a uma `class` em que todos os membros são declarados como `public`.

Funções membro embutidas

- Em C++, a regra usual de programação é definir (implementar) as funções membro fora da declaração da classe, ao contrário de algumas outras linguagens (p. ex., Python):

```
class MyClass {
 void func(int par); // Declaração
};
void MyClass::func(int par) // Definição (implementação)
{
 ...
}
```
- Também é possível, embora deva ser utilizado com moderação (ver condições abaixo), declarar e definir simultaneamente a função membro na própria declaração da classe:

```
class MyClass {
 void func(int par) { ... } // Declaração e definição
};
```
- Funções membro definidas dentro da declaração da classe são automaticamente consideradas como embutidas, o que é equivalente a precedê-las da palavra chave `inline`:

```
class MyClass {
 inline void func(int par) { ... }
};
```
- Ao declarar uma função como embutida (`inline`), o compilador preferencialmente não vai criá-la como uma função independente, com seu próprio escopo de execução, mas sim substituir toda chamada a essa “função” pela repetição do código da sua definição.
  - Declarar funções membro pequenas (1-2 linhas de código) como embutidas, especialmente se elas forem frequentemente utilizadas, pode acelerar a execução do programa, pois evita a troca de escopo cada vez que a função membro é chamada.
  - Declarar funções membro mais complexas (>2 linhas de código, com variáveis locais, incluindo laços, etc.) como embutidas pode tornar o programa executável muito grande, em função da repetição do mesmo código a cada chamada da função membro.

Mecanismos de comunicação das funções membro (métodos):

- Parâmetros de entrada (iguais às das funções normais).
- Parâmetro de retorno (igual às das funções normais).
- Objeto sobre o qual a função membro atua, cujos dados são compartilhados entre todas as chamadas de função membro do mesmo objeto.
  - Só existe nas funções membro.
  - Pode ser visto, no escopo da função membro, como um dado global compartilhado entre todas as funções membro da mesma classe, quando atuarem no mesmo objeto.

|                                                                                                              |                                                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class MyC { private:     int k; public:     void f1(int P) {k = P+1;}     int f2() {return k;} };</pre> | <pre>MyC X,Y;          // k de X,Y são lixo  // Sem nenhuma comunicação cout &lt;&lt; Y.f2(); // Imprime lixo  // Comunicação entre f1 e f2 X.f1(7);          // k de X recebe 8 cout &lt;&lt; X.f2(); // Imprime 8</pre> |
|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- Em toda função membro, está definido o ponteiro `this`, constante (não pode ser alterado) que aponta para o (contém o endereço do) objeto sobre o qual a função membro atua.

|                                                                                                                                                       |                                                                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class MyC { private:     int k; public:     void print() {         cout &lt;&lt; "this=" &lt;&lt; this             &lt;&lt; endl;     } };</pre> | <pre>MyC X,Y;  // Imprime mesmo valor nas 2 vezes cout &lt;&lt; "endX=" &lt;&lt; &amp;X &lt;&lt; endl; X.print(); // Imprime mesmo valor nas 2 vezes, // diferente dos valores anteriores cout &lt;&lt; "endY=" &lt;&lt; &amp;Y &lt;&lt; endl; Y.print();</pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- Ao definir métodos, deve-se indicar quando uma dessas grandezas é constante:
 

```
const <TIPO_RETORNO> metodo(const <PARAMETRO>) const
```

  - O primeiro `const` indica que o objeto retornado não pode ser modificado por quem o receber. Normalmente, só é usado quando o retorno é por referência, com `&`, e o objeto original não pode ser alterado.
  - O segundo `const` indica que o método não pode alterar o parâmetro. Geralmente, só quando o parâmetro é passado por referência, com `&`.
  - O terceiro `const` indica que o método não pode alterar o objeto no qual é chamado (`*this`).
- Objetos "grandes" são geralmente passados por referência, mesmo quando não se pretende alterá-los (usar o `const` nesse caso):
  - `metodo(Classe_Grande X):` NÃO OTIMIZADO: faz cópia desnecessária de X
  - `metodo(Classe_Grande &X):` OK, quando precisa alterar X
  - `metodo(const Classe_Grande &X):` OK, quando não pode alterar X

Métodos de consulta e de alteração

- Prever as funções de consulta (ex.: `get____` ) e de alteração de valores (Ex.: `set____` ).
- Muitas funções de consulta são pequenas e, portanto, declaradas como embutidas.
- As funções `set____` (e algumas `get____` ) muitas vezes devem checar parâmetros.

### Amigas (friend) de uma classe

- As funções amigas de uma classe são funções clássicas que, apesar de não serem funções membro da classe, poderão acessar os seus membros privados.

```
class MyClass {
 ...
 friend myFunc(int i); // myFunc não é membro de MyClass
};
```

- Se uma classe B é amiga da classe A, os membros da classe B podem acessar os membros privados da classe A.

```
class A {
 ...
 friend class B;
};
```

A recíproca não é verdadeira, a não ser que A também seja declarada como amiga de B.

# Construtores e destrutores

- Construtores e destrutores são métodos especiais que sempre são chamados automaticamente quando um objeto da classe é criado ou deixa de existir.
  - Classe `MyClass`: construtor denominado `MyClass`
  - Classe `MyClass`: destrutor denominado `~MyClass`
  - Pode haver mais de um construtor, desde que cada construtor seja diferente de todos os demais quanto ao tipo e/ou ao número de parâmetros.
  - Só pode haver um destrutor.
- As funções construtoras de uma classe podem ser chamadas explicitamente no código para criar variáveis sem nome (temporárias) dessa classe:

```
// Imprime uma string temporária criada com 20 caracteres #
// Utiliza um dos construtores da classe string
cout << string(20, '#') << endl;
// Soma 2 números complexos constantes
// Utiliza construtor da classe Complex com 2 parâmetros double
Complex C = Complex(1.0, -1.0) + Complex(0.0, 3.14);
```

**ATENÇÃO:** Ao chamar uma função construtora dentro de uma função membro de uma classe, isso **não** fará com que os dados do objeto em cima do qual a função membro está sendo chamada (`*this`) passem a ter os valores iniciais: apenas criará uma variável temporária que não será usada para nada e logo em seguida será destruída.

|                                                                                   |                                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class MyC { private:     int K; public:     MyC();     void func(); };</pre> | <pre>// Construtor: inicializa K com -1 MyC::MyC(): K(-1) {}  // Metodo da classe void MyC::func() {     MyC(); // Cria variável sem nome inútil     ...    // Nao faz this-&gt;K receber -1</pre> |
|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- Em programas normais, a função destrutora de uma classe não deve ser chamada explicitamente no código.
  - Quando você tem necessidade de um método que libera os recursos do objeto e que possa ser chamado durante a execução do programa, e não apenas quando o objeto for destruído, pode ser criado um método para isso (tradicionalmente denominado `clear`) e, para evitar repetição de código, chamá-lo na definição do destrutor:

|                                                                                                             |                                                                                                                           |                                                                    |
|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <pre>class Vetor { private:     int N;     double* x; public:     ~Vetor();     void clear();     ...</pre> | <pre>void Vetor::clear() {     // Libera memória     delete[] x;     // Zera conteúdo     N = 0;     x = nullptr; }</pre> | <pre>Vetor::~~Vetor () {     // Chama "clear"     clear(); }</pre> |
|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|

- Os construtores de uma classe devem utilizar e referenciar os construtores apropriados das classes dos objetos dos quais a classe é composta:
  - Na declaração da classe (no `.h`), quando o parâmetro de inicialização for único para todos os construtores e não depender dos parâmetros do construtor. A sintaxe é similar à inicialização de uma variável clássica: `nome(param)` ou `nome=param`.
  - Na definição do construtor (no `.cpp`) quando o parâmetro de inicialização for específico ou depender dos parâmetros daquele construtor. A sintaxe é, após os parâmetros do

construtor, acrescentar um : e a lista de construtores das classe base que devem ser utilizados, separados por vírgulas:

|                                                                                                                                |                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Base { private:     int P1;     double P2(0.0); public:     Base(int I); };  Base::Base(int I): P1(I) { ... }</pre> | <pre>class Deriv { private:     Base B;     int P3; public:     Deriv(int I, int J); };  Deriv::Deriv(int I, int J): B(I),P3(J) { ... }</pre> |
|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|

**ATENÇÃO:** Todo construtor, exceto raras exceções, deve indicar quais construtores das classes base devem ser utilizados para criar cada um dos dados membros dos objetos da classe, usando a sintaxe específica para isso e não inicializando com lixo e depois fazendo atribuição.

| CLASSE                                                                 | NÃO FAÇA                                                                                 | FAÇA                                                 |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------|
| <pre>class Vetor { private:     int N;     double* x;     ... };</pre> | <pre>Vetor::Vetor() // Não inicializa N e x (LIXO) {     N = 0;     x = nullptr; }</pre> | <pre>Vetor::Vetor() :     N(0), x(nullptr) { }</pre> |

- Um construtor de uma classe, em vez de repetir a inicialização dos dados membros que seja a mesma inicialização de outro construtor, pode delegar a inicialização, chamando esse outro construtor.

| DADO QUE                                             | EM VEZ DE                                                         | PODE FAZER                                               |
|------------------------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------|
| <pre>Vetor::Vetor() :     N(0), x(nullptr) { }</pre> | <pre>Vetor::Vetor( ... ) :     N(0), x(nullptr) {     ... }</pre> | <pre>Vetor::Vetor( ... ) :     Vetor() {     ... }</pre> |

## Tipos de construtores

Construtor default:

- É chamado quando o(s) objeto(s) é(são) criado(s) de maneira estática ou dinâmica sem nenhum parâmetro de inicialização:

```
MyClass A; // Chama construtor default 1 vez
MyClass X[10]; // Chama construtor default 10 vezes
ptr = new MyClass; // Chama construtor default 1 vez
ptr = new MyClass[10]; // Chama construtor default 10 vezes
```

- A função que implementa o construtor default não tem nenhum parâmetro:

```
MyClass::MyClass():__ { ... }
```

- Quase sempre existirá um construtor default:
  - Se você não declarar nenhum construtor de qualquer tipo, o compilador criará um construtor default padronizado. O comportamento desse construtor default padronizado é chamar o construtor default das classes base. Se os dados membro são de tipos padrões de C++ (int, float, ponteiros, etc.), o construtor default dessas classes é não fazer nenhum tipo de inicialização, deixando todos os dados com valor aleatório (lixo), o que **não é admissível** caso a classe tenha dados membro do tipo ponteiro.
  - Caso você crie algum construtor de qualquer tipo, o compilador não criará o construtor default padronizado.
  - Para impedir a criação do construtor default padronizado, você pode declará-lo (no .h) usando a notação:

```
MyClass() = delete;
```
  - Toda classe que utiliza alocação dinâmica de memória tem **obrigatoriamente** que prever um construtor default próprio, que não seja o construtor default padronizado.
  - Classes sem construtor default não permitem a criação de objetos sem inicialização.

Construtor por cópia:

- É chamado quando o objeto é inicializado como sendo uma cópia de outro objeto da mesma classe que tem nome (não é temporário):

```
MyClass A;
MyClass B(A); ou MyClass B=A;
ptr = new MyClass(A);
```

- O construtor por cópia também é chamado ao passar por cópia um parâmetro para uma função que é um objeto com nome (não é temporário) da classe:

```
void myFunc(MyClass X):__ { ... } // X passado por cópia
MyClass Z;
myFunc(Z); // X criado como cópia de Z
```

- A função que implementa o construtor por cópia recebe um único parâmetro, que é sempre uma referência (quase sempre constante) a outro objeto da mesma classe.

```
MyClass::MyClass(const MyClass& X): __ { ... };
```

- Sempre existirá um construtor por cópia:
  - Se você não fizer um, o compilador criará um construtor por cópia padronizado. O comportamento desse construtor por cópia padronizado é copiar todos os valores byte a byte, o que **não é admissível** caso a classe tenha algum dado do tipo ponteiro com alocação dinâmica de memória, pois serão dois objetos apontando para a mesma área, e não uma cópia do objeto original.

- Para impedir a criação do construtor por cópia padronizado, você pode declará-lo (no .h) usando a notação:  

```
MyClass(const MyClass & X) = delete;
```
- Toda classe que utiliza alocação dinâmica de memória tem **obrigatoriamente** que prever um construtor por cópia próprio, que não seja o construtor por cópia padronizado.
- Em classes sem alocação dinâmica de memória, é possível não programar um construtor por cópia explicitamente, pois o construtor padronizado que será criado deve fazer exatamente aquilo que se espera que seja feito (copiar todos os valores dos dados).
- Classes sem construtor por cópia, o que só acontece se for suprimida explicitamente a criação do construtor por cópia padronizado (=delete), não permitem a criação de objetos que sejam cópias de outros objetos nem a passagem por cópia de objetos dessa classe como parâmetros de funções.

Construtor por movimento:

- O construtor por movimento tem a mesma função que o construtor por cópia: criar um novo objeto igual a um objeto existente. A diferença entre eles só faz sentido para objetos que envolvam alocação de recursos: para outros objetos, **não é necessário nem recomendável** definir um construtor por movimento diferente do construtor por cópia.
- Enquanto no construtor por cópia tanto o objeto original quanto o novo objeto devem permanecer existindo de maneira independente após a execução do construtor, o que exige a alocação de novos recursos específicos para o novo objeto, no construtor por movimento o objeto original é temporário e vai desaparecer após a execução do construtor. Por essa razão, os recursos do objeto original podem ser movidos (“roubados”) para o novo objeto, sem necessidade de fazer uma nova alocação.
- É chamado quando o objeto é inicializado como sendo uma cópia de outro objeto sem nome (temporário) da mesma classe:

|                                                               |                                                                 |
|---------------------------------------------------------------|-----------------------------------------------------------------|
| <pre>MyClass A,B;<br/>MyClass C(A+B); ou MyClass C=A+B;</pre> | <pre>MyClass func(...);<br/>ptr = new MyClass(func(...));</pre> |
|---------------------------------------------------------------|-----------------------------------------------------------------|

- O construtor por movimento também é chamado ao retornar um objeto local como valor de retorno de uma função:  

```
MyClass myFunc(void) {
 MyClass prov;
 ...
 return prov; // prov desaparece após return; pode ser movido
}
// A variável sem nome que contém o valor de retorno da função
// é criada com o construtor por movimento
cout << myFunc();
```
- A função que implementa o construtor por movimento recebe um único parâmetro que é uma referência (não constante) a outro objeto temporário (sem nome) da mesma classe. As referências a objetos temporários são representadas por um duplo && entre o tipo e o nome (por exemplo, `MyClass&& X`) para diferenciar das referências a objetos em geral, que são representadas com um único & (por exemplo, `MyClass& X`):  

```
MyClass::MyClass(MyClass&& X) noexcept : __ { ... };
```
- Normalmente, a função que implementa o construtor por movimento é seguida do especificador `noexcept`, indicando que ela nunca gerará uma exceção ao ser executada.
  - A utilização do especificador `noexcept` permite que o compilador faça otimizações na velocidade de execução do programa.
  - Uma função com especificador `noexcept` não pode chamar nenhuma outra função que possa gerar exceção: alocar memória, abrir ou fazer operações com arquivos, etc., o que normalmente não é o caso em construtores por movimento.



- O especificador `noexcept` pode ser acrescentado a outros construtores e a qualquer função que não gere exceção, embora a possibilidade de que isso gere alguma otimização é maior apenas no construtor e no operador de atribuição por movimento.
- No objeto original, é preciso indicar que ele não possui mais o recurso para que ele não seja liberado pelo destrutor do objeto temporário e o novo objeto não fique com um recurso inválido. Isso pode ser feito de duas formas:
  - Fazer o ponteiro (e os demais dados) do novo objeto ser igual ao ponteiro do objeto original e, em seguida, atribuir ao ponteiro original um valor nulo (`nullptr`); ou
  - Inicializar o novo objeto com valores nulos e, em seguida, permutar (`swap`) esses valores com os do objeto original.

| Exemplos da diferença entre construtor por cópia e por movimento (2 opções)                                                           |                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Classe                                                                                                                                | Construtor por cópia                                                                                                                                                                      |
| <pre>class Vetor { private:     int N;     double* x;     ... };</pre>                                                                | <pre>Vetor::Vetor(const Vetor&amp; V) :     N(V.N),     x(N&gt;0 ? new double[N] : nullptr) { // Cópia (não faz nada se N==0)     for (int i=0; i&lt;N; ++i)         x[i]=V.x[i]; }</pre> |
| Construtor por movimento (opção 1)                                                                                                    | Construtor por movimento (opção 2)                                                                                                                                                        |
| <pre>Vetor::Vetor(Vetor&amp;&amp; V) noexcept:     N(V.N), x(V.x) // Cópia { // Zera o temporário     V.N=0;     V.x=nullptr; }</pre> | <pre>Vetor::Vetor(Vetor&amp;&amp; V) noexcept:     N(0), x(nullptr) // Zera { // Permuta os dados     swap(N,V.N);     swap(x,V.x); }</pre>                                               |

- Sempre existirá um construtor por movimento:
  - Se você não fizer um, o compilador criará um construtor por movimento padronizado idêntico ao construtor por cópia, o que **não é admissível** caso a classe tenha algum dado do tipo ponteiro e esteja sendo utilizado o construtor por cópia padronizado.
  - Para impedir a criação do construtor por movimento padronizado, você pode declará-lo (no `.h`) usando a notação:
 

```
MyClass(const MyClass && X) = delete;
```
  - É recomendável que toda classe que utiliza alocação dinâmica de memória preveja um construtor por movimento próprio, que não seja o construtor por movimento padronizado, para evitar alocações desnecessárias para copiar objetos temporários.

#### Construtores específicos:

- Todos os outros construtores que não sejam o construtor default, por cópia ou por movimento são chamados de construtores específicos.
- Os diferentes construtores específicos são chamados quando o objeto é inicializado tendo como parâmetro um objeto de outra classe ou mais de um parâmetro de inicialização. A escolha de qual construtor específico será executado depende do tipo e da quantidade de parâmetros de inicialização:

```
MyClass::MyClass() :__ { ... }; // Default
MyClass::MyClass(int I) :__ { ... }; // Especifico 1
MyClass::MyClass(int I, double D) :__ { ... }; // Especifico 2
MyClass::MyClass(MyClass X, int I) :__ { ... }; // Especifico 3
```

```
int k;
MyClass B,C; // Construtor default
MyClass A(k); // Construtor especifico 1
MyClass A(k,2.71); // Construtor especifico 2
```

```
MyClass A(B,k); // Construtor específico 3
```

- Os construtores específicos com um único parâmetro de outra classe também podem ser chamados para promover um objeto, ou seja, converter o objeto da outra classe para criar um objeto sem nome (temporário) da classe do construtor:

```
// Construtor específico com um único parâmetro de outra classe
// Pode ser usado para promover
```

```
MyClass::MyClass(int I):__ { ... };
void myFunc(MyClass X, MyClass Y) { ... }
```

```
MyClass M;
myFunc(M,1); // Será chamado construtor para converter o 1
```

- Para impedir que um construtor específico com um único parâmetro de outra classe seja utilizado em promoções, deve-se utilizar a palavra-chave `explicit` antes da declaração do construtor (no `.h`), o que fará com que ele só seja chamado se for explicitamente incluído no código:

```
// Não é usado em promoções
explicit MyClass::MyClass(int I):__ { ... };
```

```
myFunc(M,1); // Erro de compilação
myFunc(M,MyClass(1)); // OK
```

# Sobrecarga de operadores

- C++ permite usar operadores padrões da linguagem para executar operações não só com os tipos fundamentais, mas também com classes criadas.

|                                    |                                        |
|------------------------------------|----------------------------------------|
| <pre>int a,b,c; ... a = b+c;</pre> | <pre>MyClass A,B,C; ... A = B+C;</pre> |
|------------------------------------|----------------------------------------|

- Para sobrecarregar um operador e usá-lo com classes, declaramos *funções operador*, que são funções cujos nomes são a palavra-chave `operator` seguida pelo símbolo do operador que queremos sobrecarregar.

o `<TIPO_RETORNO> operator<SÍMBOLO>(<PARAMETROS>) {...}`, `<SÍMBOLO>` = símbolo do operador

| OPERADORES QUE PODEM SER SOBRECARREGADOS |    |    |    |    |    |    |    |     |    |     |        |       |          |
|------------------------------------------|----|----|----|----|----|----|----|-----|----|-----|--------|-------|----------|
| +                                        | -  | *  | /  | =  | <  | >  | += | -=  | *= | /=  | <<     | >>    | <=>      |
| >>=                                      | == | != | <= | >= | ++ | -- | %  | &   | ^  | !   |        | ~     | &=       |
| ^=                                       | =  | && |    | %= | [] | () | ,  | ->* | -> | new | delete | new[] | delete[] |

- O operador sobrecarregado pode ser utilizado através do nome completo da função membro (`operator<SÍMBOLO>`) ou através do símbolo do operador (`<SÍMBOLO>`), de forma indistinta.

| USANDO SÍMBOLO DO OPERADOR | USANDO NOME DA FUNÇÃO                       |
|----------------------------|---------------------------------------------|
| <code>... B+C ...</code>   | <code>... B.operator+(C) ...</code>         |
| <code>A = B;</code>        | <code>A.operator=(B);</code>                |
| <code>A = B+C;</code>      | <code>A.operator=( B.operator+(C) );</code> |

- Os operadores devem ser sobrecarregados apenas com o sentido usual ou próximo. Evite, por exemplo, sobrecarregar o operador '+' para fazer multiplicações!
- Para alguns operadores, existem duas maneiras de sobrecarregá-los: como uma função membro C++ ou como uma função clássica C. Seu uso geralmente é equivalente, exceto nos casos listados a seguir. No entanto, é importante lembrar que funções que não são membros de uma classe não podem acessar os membros privados ou protegidos dessa classe, a menos que a função seja declarada como `friend` da classe.

| FORMA DE DECLARAÇÃO DAS FUNÇÕES OPERADOR                                         |                                                                                                |                                              |                                             |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------|---------------------------------------------|
| a é um objeto da classe A, b é um objeto da classe B e c é um objeto da classe C |                                                                                                |                                              |                                             |
| Expressão                                                                        | Operador                                                                                       | Função membro                                | Função clássica                             |
| <code>&lt;SÍMBOLO&gt;a</code>                                                    | <code>+ - * &amp; ! ~ ++ --</code>                                                             | <code>A::operator&lt;SÍMBOLO&gt;()</code>    | <code>operator&lt;SÍMBOLO&gt;(A)</code>     |
| <code>a&lt;SÍMBOLO&gt;</code>                                                    | <code>++ --</code>                                                                             | <code>A::operator&lt;SÍMBOLO&gt;(int)</code> | <code>operator&lt;SÍMBOLO&gt;(A,int)</code> |
| <code>a&lt;SÍMBOLO&gt;b</code>                                                   | <code>+ - * / % ^ &amp;   &lt; &gt; == != &lt;= &gt;= &lt;&lt; &gt;&gt; &amp;&amp;    ,</code> | <code>A::operator&lt;SÍMBOLO&gt;(B)</code>   | <code>operator&lt;SÍMBOLO&gt;(A,B)</code>   |
| <code>a&lt;SÍMBOLO&gt;b</code>                                                   | <code>= += -= *= /= %= ^= &amp;=  = &lt;=&gt; &gt;&gt;=</code>                                 | <code>A::operator&lt;SÍMBOLO&gt;(B)</code>   | -                                           |
| <code>a[b]</code>                                                                | <code>[]</code>                                                                                | <code>A::operator[] (B)</code>               | -                                           |
| <code>a(b, c,...)</code>                                                         | <code>()</code>                                                                                | <code>A::operator() (B,C,...)</code>         | -                                           |
| <code>a-&gt;x</code>                                                             | <code>-&gt;</code>                                                                             | <code>A::operator-&gt;()</code>              | -                                           |

- Operadores binários com parâmetros do mesmo tipo (soma, menor que, etc.) normalmente devem ser sobrecarregados como funções clássicas, e não funções membros, para permitir que ambos os parâmetros possam ser convertidos, caso necessário:

| MyClass A,B,C;                               |                                              |                                                              |
|----------------------------------------------|----------------------------------------------|--------------------------------------------------------------|
| // MyClass + MyClass<br>A = B+C;             | // MyClass + int<br>A = B+1;                 | // int + MyClass<br>A = 1+C;                                 |
| // OK função membro<br>A = B.operator+(C);   | // OK se há conversor<br>A = B.operator+(1); | // Erro, mesmo se há conversor<br><b>A = 1.operator+(C);</b> |
| // OK função clássica<br>A = operator+(B,C); | // OK se há conversor<br>A = operator+(B,1); | // OK se há conversor<br>A = operator+(1,C);                 |

- Os operadores >> e <<, quando estiverem sendo usados como operações de inserção (impressão) e extração (leitura) em streams, devem ser sobrecarregados como funções clássicas friend, não funções membro:
  - friend ostream &operator<<(ostream &X, const \_\_\_\_ &M);
  - friend istream &operator>>(istream &X, \_\_\_\_ &M);
 Isso porque, caso fossem sobrecarregados como funções membros, teriam que pertencer à classe que está à esquerda do operador, que é uma stream (cin, cout, etc.). Essas são classes da linguagem padrão, que o programador não pode alterar para acrescentar esse novo membro.
- As funções clássicas que implementam os operadores de inserção e extração devem retornar por referência a mesma stream que receberam, também por referência, como seu primeiro parâmetro. Isso porque muitas vezes esses operadores são encadeados, fazendo várias operações de inserção ou extração em uma mesma linha de código. Para isso, a segunda chamada da função vai receber como primeiro parâmetro o valor retornado pela primeira chamada da função.

| CÓDIGO ORIGINAL          | CÓDIGO NO QUAL SERÁ CONVERTIDO              |
|--------------------------|---------------------------------------------|
| cout << "Hello" << endl; | operator<<(operator<<(cout,"Hello"), endl); |

- Caso os operadores >> e << estejam sendo usados como operadores de deslocamento à direita e à esquerda, e não como operadores de inserção e extração em streams, podem ser sobrecarregados como funções membro.

Operador de atribuição (`operator=`):

- O operador de atribuição só pode ser sobrecarregado como função membro.
- Sempre existirá um operador de atribuição:
  - Se você não fizer um, o compilador criará um operador de atribuição padronizado. O comportamento desse `operator=` padronizado é copiar todos os valores byte a byte, o que **não é admissível** caso a classe tenha algum dado do tipo ponteiro, pois serão dois objetos apontando para a mesma área, e não uma cópia do objeto original.
  - Para impedir a criação do `operator=` padronizado, você pode deletá-lo (no `.h`):  
`Classe& Classe::operator=(const Classe & X) = delete;`
  - Toda classe que utiliza alocação dinâmica de memória tem **obrigatoriamente** que prever um operador de atribuição próprio, que não seja o `operator=` padronizado.

- De forma similar ao construtor por cópia e por movimento, também existem o operador de atribuição por cópia e por movimento:

```
Classe& Classe::operator=(const Classe& X); // Cópia
Classe& Classe::operator=(Classe&& X) noexcept; // Movimento
```

A diferença entre eles, da mesma forma que entre os construtores, é que, quando o parâmetro é um objeto temporário, seus recursos alocados podem ser movidos (“roubados”) em vez de copiados para outra área de memória especialmente alocada para isso.

- Normalmente, a função que implementa o `operator=` por movimento é seguida do especificador `noexcept`, pelas mesmas razões que o construtor por movimento.
- Para permitir o encadeamento de atribuições (`A=B=C;`), a função `operator=` (tanto por cópia quanto por movimento) deve retornar uma referência ao objeto atribuído (`*this`):

```
Classe& Classe::operator=(...) {
 ...
 return *this;
}
```

- Normalmente se inclui na definição do `operator=` por cópia e por movimento uma proteção contra autoatribuição de variáveis (`A=A;` ou `A=move(A);`):

```
Classe& Classe::operator=(const Classe& X) {
 if (this == &X) return *this; // Encerra e já retorna
 // Aqui vai todo o código do operador por cópia
 ...
 return *this;
}
```

```
Classe& Classe::operator=(Classe&& X) noexcept {
 if (this == &X) return *this; // Encerra e já retorna
 // Aqui vai todo o código do operador por movimento
 ...
 return *this;
}
```

- O código do `operator=` por cópia, caso passe no teste de que não se trata de autoatribuição, geralmente é composto por código similar ao do destrutor (limpar o conteúdo anterior) seguido por código similar ao do construtor por cópia (alocar e fazer a cópia).
- O código do `operator=` por movimento, caso passe no teste de que não se trata de autoatribuição, geralmente é similar ao código do destrutor (limpar o conteúdo anterior) seguido de código similar ao do construtor por movimento. Da mesma forma que no construtor por movimento, isso pode ser feito de duas formas:
  - Fazer o ponteiro (e os demais dados) do objeto sendo atribuído (lado esquerdo do `operator=`) ser igual ao ponteiro do objeto original (lado direito do `operator=`) e, em seguida, atribuir ao ponteiro original um valor nulo (`nullptr`); ou
  - Inicializar o objeto sendo atribuído com valores nulos e, em seguida, permutar (`swap`) esses valores com os do objeto original.

| Exemplos da diferença entre operador de atribuição por cópia e operador de atribuição por movimento (2 opções)                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Classe                                                                                                                                                                                                                                                                                                                          | Operador de atribuição por cópia                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <pre>class Vetor { private:     int N;     double* x;     ... };</pre>                                                                                                                                                                                                                                                          | <pre>Vetor&amp; Vetor::operator=     (const Vetor&amp; V) {     // Retorna se autoatribuição     if (this == &amp;X) return *this;      // Limpa conteúdo anterior     delete[] x;      // Aloca nova área     N = V.N;     if (N&gt;0) x=new double[N];     else x=nullptr;     // Alternativa usando ternário     // x=(N&gt;0?new dou- ble[N]:nullptr);      // Copia valores     for (int i=0; i&lt;N; ++i)         x[i] = V.x[i];     return *this; }</pre> |
| Operador de atribuição por movimento<br>(opção 1)                                                                                                                                                                                                                                                                               | Operador de atribuição por movimento<br>(opção 2)                                                                                                                                                                                                                                                                                                                                                                                                                |
| <pre>Vetor&amp; Vetor::operator=     (Vetor&amp;&amp; V) noexcept {     // Retorna se autoatribuição     if (this == &amp;X) return *this;      // Limpa conteúdo anterior     delete[] x;      // Move conteúdo     N = V.N;     x = V.x;      // Zera o temporário     V.N = 0;     V.x = nullptr;      return *this; }</pre> | <pre>Vetor&amp; Vetor::operator=     (Vetor&amp;&amp; V) noexcept {     // Retorna se autoatribuição     if (this == &amp;X) return *this;      // Limpa conteúdo anterior     delete[] x;      // Zera o objeto atribuído     N = 0;     x = nullptr;      // Permuta os conteúdos     swap(N,V.N);     swap(x,V.x);      return *this; }</pre>                                                                                                                 |

# Classes com alocação dinâmica de memória

## Obrigatoriedades:

- Toda classe que utiliza alocação dinâmica de memória tem **obrigatoriamente** que prever:
  - Construtor default que não seja o construtor padronizado
  - Construtor por cópia que não seja o construtor padronizado
  - Destrutor
  - Sobrecarga do operador de atribuição (`operator=`) por cópia
- Embora não seja estritamente obrigatório, é **altamente recomendável** que toda classe que utiliza alocação dinâmica de memória preveja:
  - Construtor por movimento que não seja o construtor padronizado
  - Sobrecarga do operador de atribuição (`operator=`) por movimento

## Boas técnicas de programação:

- Geralmente, a alocação (`new`, `new ...[...]`) e a desalocação (`delete`, `delete[]`) de ponteiros só deve ser feita nos métodos obrigatórios das classes (construtores, destrutor e operadores de atribuição).
- Nas demais funções (outros construtores, métodos da classe ou utilização da classe no programa principal), para alterar os recursos alocados a um objeto, normalmente deve-se utilizar algum dos métodos obrigatórios que execute a modificação desejada:
  - Construtores podem delegar a inicialização a outros construtores, caso os valores iniciais dos dados membros sejam os mesmos.
  - Outras funções podem atribuir valores usando a atribuição por movimento e variáveis sem nome criadas chamando explicitamente algum construtor:

```
X = MyClass(); // Atribui valor igual ao construtor default
X = MyClass(...); // Algum construtor específico
```
- Exceto em situações muito específicas, operadores de atribuição **não podem ser utilizados** na implementação de construtores. Isso porque, normalmente, a função `operator=` primeiro faz a liberação de memória do objeto que está à esquerda (`*this`) para, em seguida, alocar memória para criar uma cópia ou mover o conteúdo da direita. Em um construtor, o objeto (`*this`) está sendo criado e, portanto, não tem conteúdo anterior. O conteúdo dos ponteiros é aleatório (ponteiro selvagem, *wild pointer*). Ao tentar liberar essa “memória”, pode ocorrer um erro de execução e travamento do programa.

| Exemplos de boa programação em classes com alocação dinâmica de memória                                                       |                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class Vetor { private:     int N;     double* x;     ... }; </pre>                                                      |                                                                                                                                                            |
| NÃO FAÇA                                                                                                                      | FAÇA                                                                                                                                                       |
| <pre> Vetor::Vetor() {     *this = Vetor(); // ERRO     GRAVE! } </pre>                                                       | <pre> Vetor::Vetor() :     N(0), x(nullptr) {} </pre>                                                                                                      |
| <pre> Vetor::Vetor(const Vetor&amp; V) {     *this = V; // ERRO GRAVE! } </pre>                                               | <pre> Vetor::Vetor(const Vetor&amp; V):     N(V.N),     x(N&gt;0 ? new double[N] : nullptr) {     for (int i=0; i&lt;N; ++i)         x[i]=V.x[i]; } </pre> |
| <pre> Vetor V; V.N = 10; V.x = new double[10]; </pre>                                                                         | <pre> // Usa construtor específico Vetor V(10); </pre>                                                                                                     |
| <pre> if ( ... ) {     Vetor V;     V.N = 0;     V.x = nullptr;     return V; } </pre>                                        | <pre> if ( ... ) {     // Usa construtor default     return Vetor(); } </pre>                                                                              |
| <pre> if ( ... ) {     Vetor V(0);     return V; } </pre>                                                                     |                                                                                                                                                            |
| <pre> Vetor A,B; ... delete[] B.x; B.N = A.N; B.x = new double[B.N]; for (int i=0; i&lt;B.N; ++i)     B.x[i] = A.x[i]; </pre> | <pre> Vetor A,B; ... // Usa operator= por cópia B = A; </pre>                                                                                              |
| <pre> Vetor A; ... delete[] A.x; A.N = 10; A.x = new double[10]; </pre>                                                       | <pre> Vetor A; ... // Usa construtor específico // e operator= por movimento A = Vetor(10); </pre>                                                         |